

♥ Like

📌 Save

🔗 Share



@thedevspaceio

CSS Grid Cheatsheet



Basic grid properties

CSS Grid is a two-dimensional layout system for creating complex responsive layouts with rows and columns.

Property	Notes
<code>display</code>	<code>grid</code> or <code>inline-grid</code> establishes a grid formatting context on the element.
<code>grid-template-columns</code>	Defines explicit column tracks (sizes, repeat, minmax, fr).
<code>grid-template-rows</code>	Defines explicit row tracks.
<code>grid-template-areas</code>	Named grid areas for readable layout placement.
<code>grid-template</code>	Shorthand for <code>grid-template-rows</code> , <code>grid-template-columns</code> , and <code>grid-template-areas</code> .
<code>gap</code>	Shorthand for <code>row-gap</code> and <code>column-gap</code> ; spacing between tracks.
<code>row-gap</code>	Controls spacing between rows.
<code>column-gap</code>	Controls spacing between columns.
<code>grid-auto-flow</code>	Controls auto-placement direction: <code>row</code> , <code>column</code> , and <code>dense</code> options.
<code>grid-auto-columns</code>	Size for implicitly created columns.
<code>grid-auto-rows</code>	Size for implicitly created rows.

Property	Notes
<code>grid-column</code>	Shorthand for <code>grid-column-start</code> / <code>grid-column-end</code> (e.g., <code>1 / -1</code> to span).
<code>grid-row</code>	Shorthand for <code>grid-row-start</code> / <code>grid-row-end</code> .
<code>grid-column-start</code>	Explicit start line for columns.
<code>grid-column-end</code>	Explicit end line for columns.
<code>grid-row-start</code>	Explicit start line for rows.
<code>grid-row-end</code>	Explicit end line for rows.
<code>grid-area</code>	Assigns an item to a named grid area or sets start/end lines.

display

Turns an element into a grid container.

css



```
1 .container {  
2   display: grid; /* or inline-grid */  
3 }
```

Grid container

grid-template-columns / rows

Define explicit columns and rows.

css



```
1 .container {  
2   display: grid;  
3   grid-template-columns: 80px 1fr 1fr;  
4   grid-template-rows: auto 50px auto;  
5   gap: 16px;  
6 }
```

1

2

3

4

5

6

7

8

9

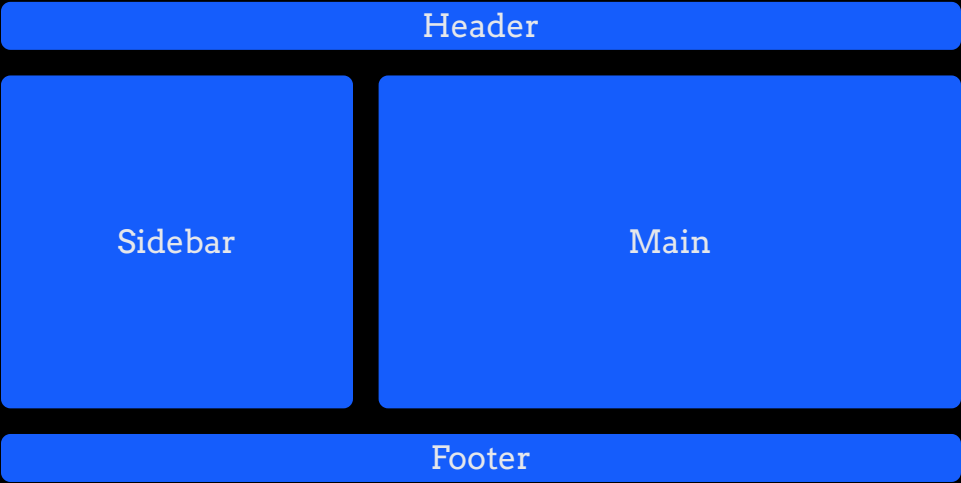
grid-template-areas

Named areas make layouts readable.

css



```
1  .container {
2    display: grid;
3    grid-template-columns: 220px 1fr; /* 2 columns */
4    grid-template-rows: auto 1fr auto; /* 3 rows */
5    grid-template-areas:
6      "header header" /* 1st row */
7      "sidebar main" /* 2nd row, sidebar left, main right */
8      "footer footer"; /* 3rd row */
9    gap: 16px;
10   min-height: 300px;
11 }
12 .header {
13   grid-area: header;
14 }
15 .sidebar {
16   grid-area: sidebar;
17 }
18 .main {
19   grid-area: main;
20 }
21 .footer {
22   grid-area: footer;
23 }
```



grid-template

Shorthand combining `grid-template-rows`, `grid-template-columns`, and `grid-template-areas`.

css



```
1 .container {  
2   grid-template:  
3     "header header" auto /* 1st row, auto height */  
4     "sidebar main" 1fr /* 2nd row, 1fr height */  
5     "footer footer" auto / 220px 1fr; /* 3rd row: auto height, 2 c  
6 }
```

gap

`gap` sets spacing between rows and columns. Shorthand for `row-gap` and `column-gap`.

css



```
1 .container {  
2   display: grid;  
3   gap: 16px; /* sets both row-gap and column-gap */  
4   /* gap: <row> <column>; */  
5   /* gap: <top> <right> <bottom> <left>; */  
6 }
```

1

2

3

1

2

3

1

2

3

row-gap / column-gap

Control spacing in one axis.

css



```
1 .container {  
2   row-gap: 12px;  
3   column-gap: 24px;  
4 }
```

1

2

3

4

5

6

grid-auto-flow

Controls how auto-placed items flow: `row` (default), `column`, and `dense` for filling gaps.

css



```
1 .container {  
2   display: grid;  
3   grid-template-columns: repeat(3, 1fr);  
4   grid-auto-flow: column; /* fills columns first */  
5 }
```

1

2

3

4

css



```
1 .container {  
2   display: grid;  
3   grid-template-columns: repeat(3, 1fr);  
4   grid-auto-flow: row; /* fills rows first */  
5 }
```

1

2

3

4

grid-auto-columns / grid-auto-rows

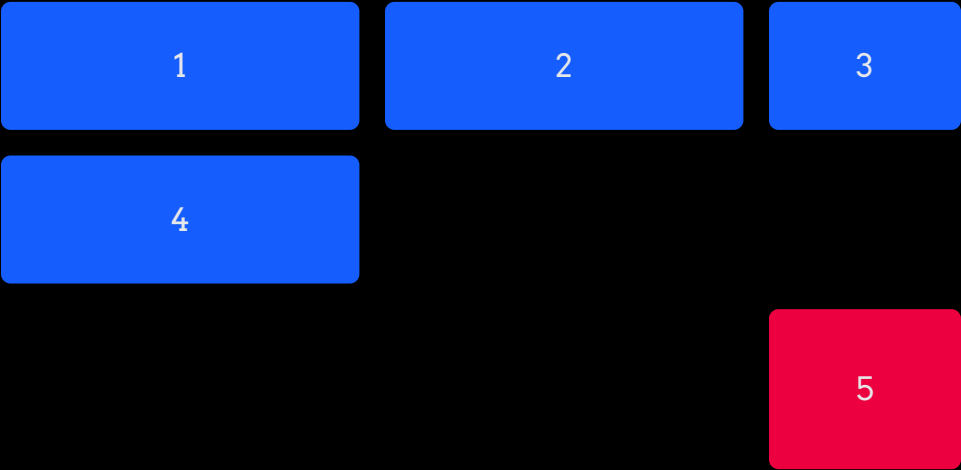
Define size for implicitly created tracks when items are placed outside the explicit grid.

css



```
1  .container {
2    display: grid;
3
4    /* Explicit grid */
5    grid-template-columns: repeat(2, 1fr);
6    grid-template-rows: repeat(2, 80px);
7
8    /* Implicit tracks */
9    grid-auto-rows: 100px;
10   grid-auto-columns: 120px;
11
12   gap: 16px;
13 }
14
15 .item-extra {
16   grid-column: 3; /* creates an implicit column */
17   grid-row: 3; /* creates an implicit row */
18 }
```

.....



grid-column / grid-row

Shorthand to set start / end lines for an item.

css



```
1  .container {
2    display: grid;
3    grid-template-columns: repeat(4, 1fr);
4    grid-template-rows: repeat(2, 1fr);
5    gap: 16px;
6  }
7  .item-full {
8    grid-column: 1 / -1; /* span full width */
9  }
10 .item-tall {
11   grid-row: 1 / 3; /* span two rows */
12 }
```

Full width

1 column

2 columns

1 column, 2
rows

1 column

1 column

1 column

grid-column-start / grid-column-end / grid-row-start / grid-row-end

Use when you need explicit control of start and end lines.

css



```
1  .container {
2    display: grid;
3    grid-template-columns: repeat(4, 1fr);
4    grid-template-rows: repeat(2, 1fr);
5    gap: 16px;
6  }
7  .card {
8    grid-column-start: 2;
9    grid-column-end: 4;
10   grid-row-start: 1;
11   grid-row-end: 2;
12 }
```

grid-area

Assign an item to a named grid area or define start/end lines with shorthand.

css



```
1  .header {
2    grid-area: header;
3  }
4  .main {
5    grid-area: main;
6  }
7
8  /* or as shorthand */
9  .box {
10   grid-area: 1 / 2 / 2 / 4; /* row-start / col-start / row-end / c
11 }
```

repeat()

Use `minmax` and `auto-fit` / `auto-fill` with `repeat()` for responsive grids.

css



```
1 .container {  
2   grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));  
3 }
```

Card 1

Card 2

Card 3